Nexus Repository Manager Implementation

Centralized Artifact Management for Multiple Development Teams

Project Type: DevOps / Artifact Repository Setup

This document describes the implementation of a centralized artifact repository using Nexus. It includes repository creation, access control configuration, artifact publishing, artifact retrieval through the Nexus REST API, and automation of application deployment.

Table of Contents

1. Project Overview
2. System Architecture
3. Nexus Installation
4. npm Repository Setup
5. Team 1 User Configuration
6. Publishing NodeJS Artifacts
7. Maven Repository Setup
8. Team 2 User Configuration
9. Publishing Java Artifacts
10. Retrieving Artifacts via Nexus API
11. Deployment Automation
12. Project Outcome

1. Project Overview

Within the company environment multiple teams were developing small applications such as NodeJS services and Java applications. Each team managed artifacts independently, which created difficulties in version control, sharing builds, and maintaining application releases.

To solve this problem a centralized artifact repository was introduced using Nexus Repository Manager. This repository allows development teams to store build artifacts in a single location while controlling access permissions for different teams and projects.

Project Objectives

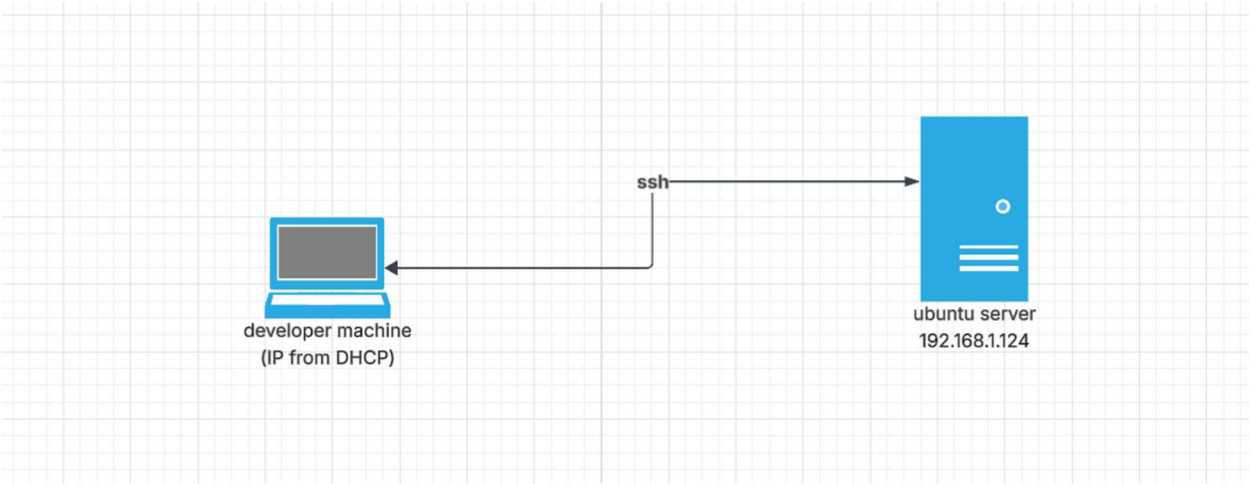
- Deploy Nexus Repository Manager on a server
- Create separate repositories for two different development projects
- Configure user permissions for each team
- Publish NodeJS and Java artifacts
- Retrieve artifacts using the Nexus REST API
- Automate application deployment using shell scripts

2. System Architecture

The implemented architecture consists of three main components:

- Nexus repository server
- Developer machines responsible for publishing artifacts
- Deployment server responsible for retrieving and running application artifacts

The Nexus REST API is used to retrieve metadata and download URLs for artifacts stored in the repository.



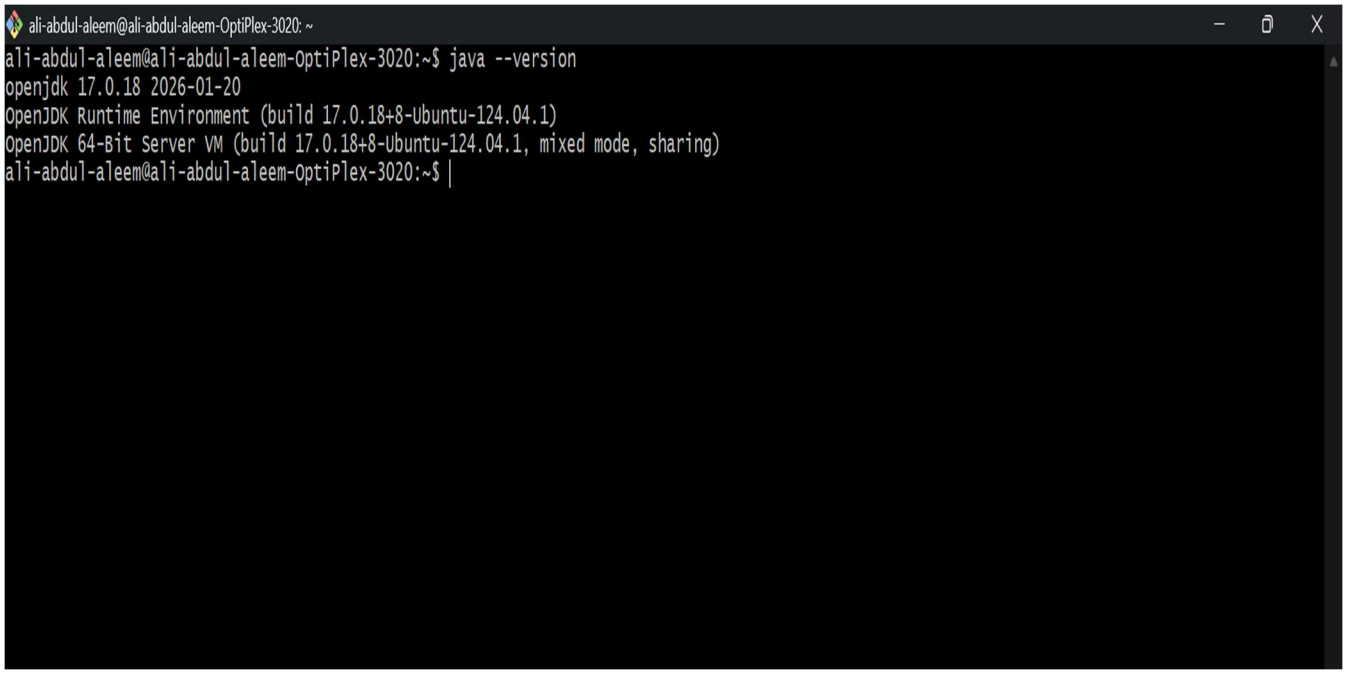
3. Nexus Installation

Nexus Repository Manager was installed on a dedicated server to act as the centralized artifact storage platform for development teams.

Installation Steps

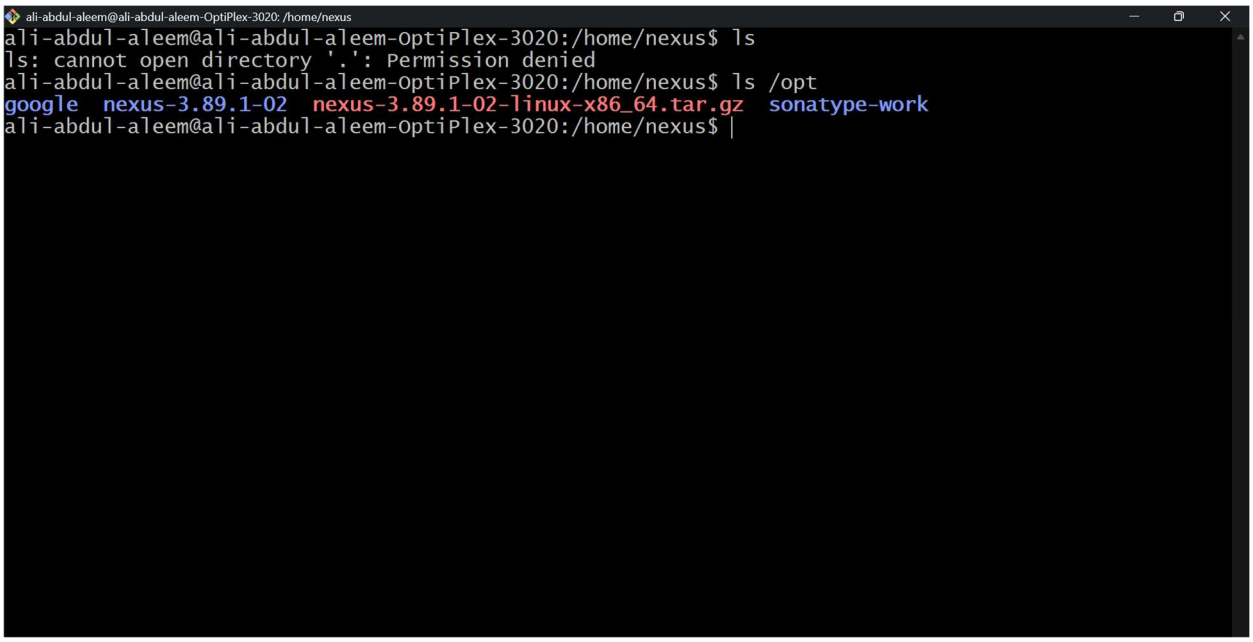
1. Install Java Runtime Environment
2. Download Nexus Repository Manager package
3. Extract the Nexus archive
4. Start the Nexus service
5. Access the Nexus web interface through port **8081**

I installed java runtime version 17 for compatibility with nexus version 3.89



Nexus was downloaded in the /opt directory.

The downloaded file was unzipped and two directories were extracted as shown in the screenshot (**nexus-3.89.1-02** and **sonatype-work**).



Next, I created a dedicated user for Nexus, gave it the required permissions for the extracted directories and configured Nexus to run as that user. The configuration file can be found in the ‘bin’ directory (/opt/nexus-3.89.1-02/bin/nexus).

```
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020: /opt
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020:/opt$ ll
total 454268
drwxr-xr-x  5 root  root       4096 Feb 25 13:32 ./
drwxr-xr-x 23 root  root       4096 Feb 25 23:40 ../
drwxr-xr-x  3 root  root       4096 Feb 25 13:01 google/
drwxr-xr-x  6 nexus nexus      4096 Feb 25 13:32 nexus-3.89.1-02/
-rw-r--r--  1 root  root  465144623 Feb 11 23:16 nexus-3.89.1-02-linux-x86_64.tar.gz
drwxr-xr-x  3 nexus nexus      4096 Feb 11 05:29 sonatype-work/
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020:/opt$ |
```

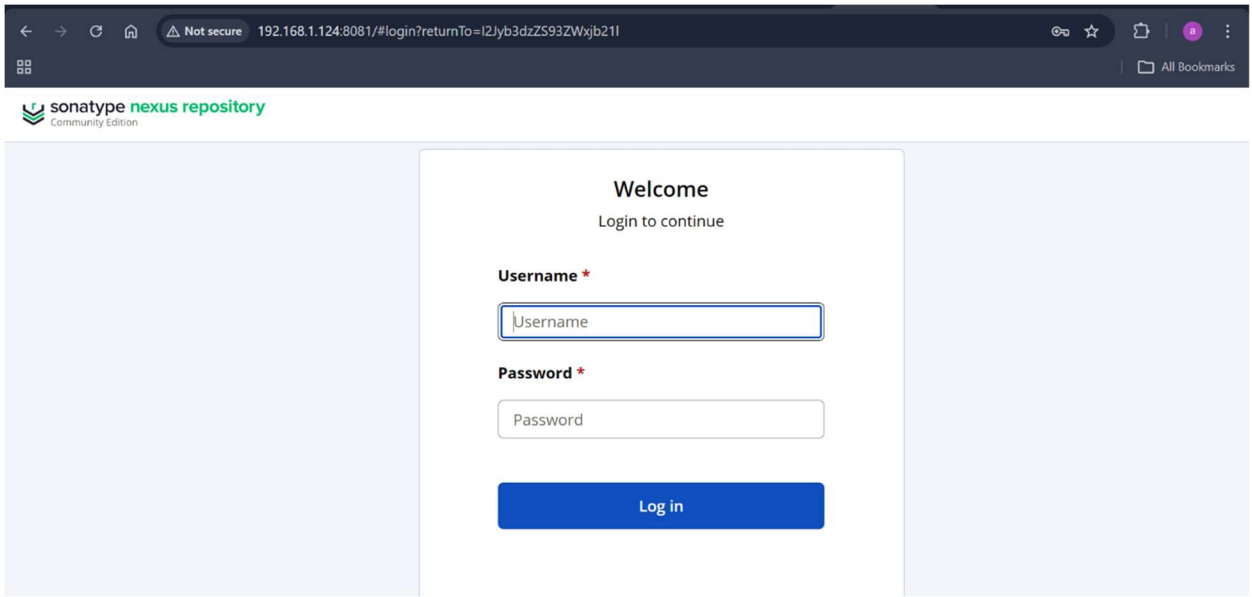
```
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020: /opt
bootJar=$(realpath $bootJar)
cd '../' || exit 1
PARENT=`pwd`
EMBEDDED_JDK="$(realpath jdk/temurin_*_linux_x86_64/jdk-21.*)"

# user to execute as; optional but recommended to set
run_as_user='nexus'

# load optional configuration
rc_file="$HOME/nexus.rc"
if [ -f "$rc_file" ]; then
  . "$rc_file"
fi

# detect if execute as root user
run_as_root=true
user_id=`id -u`
user_name=`id -u -n`
if [ -z "$run_as_user" ] && [ $user_id -ne 0 ]; then
  run_as_root=false
elif [ -n "$run_as_user" ] && [ "$run_as_user" != 'root' ]; then
  run_as_root=false
fi
```

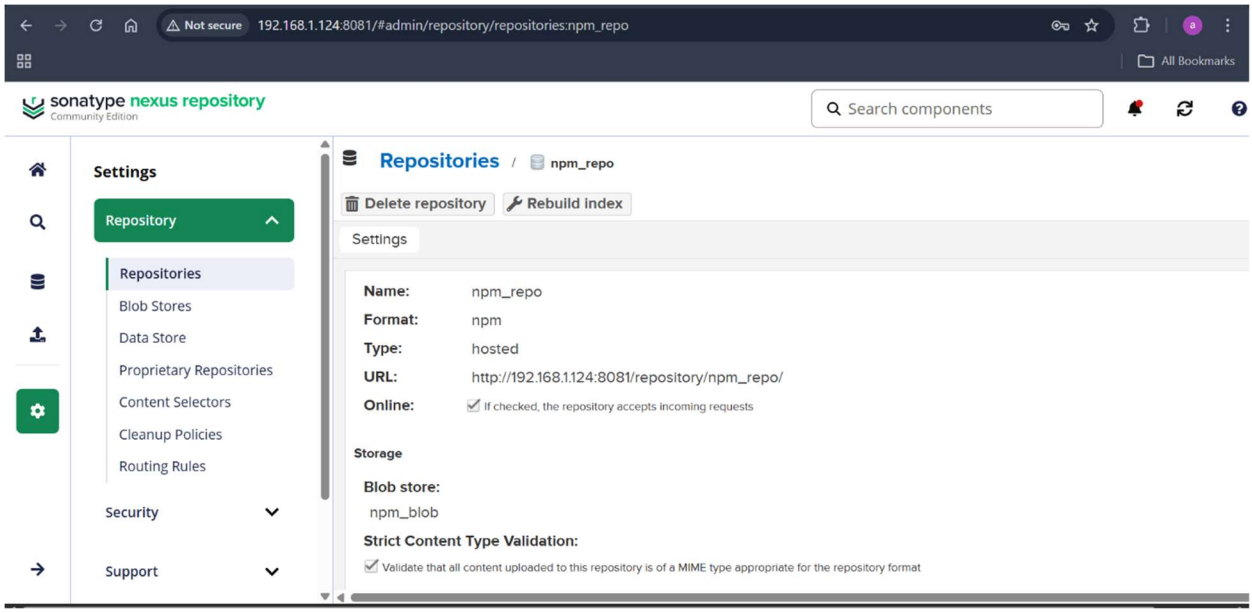
The service was started using the command '/opt/nexus-3.89.1-02/bin/nexus start' and was listening on port 8081.

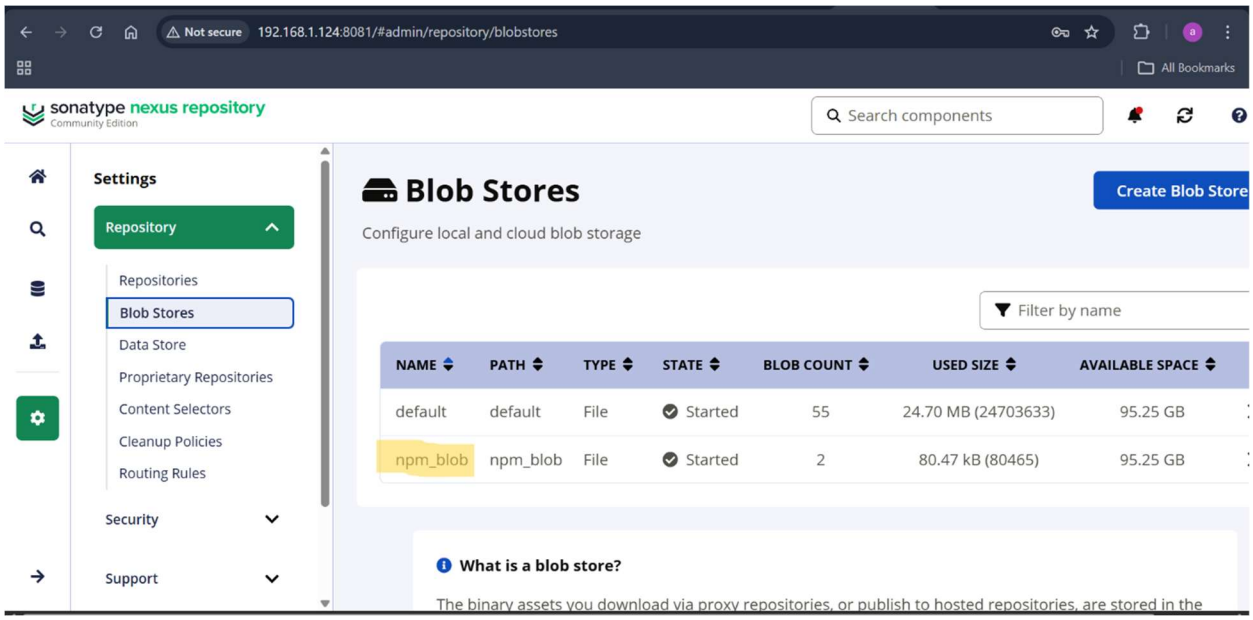


4. npm Repository Setup

A hosted npm repository was created in Nexus to store NodeJS application packages.

A dedicated **blob store** was also created to isolate npm artifacts from other repository types.



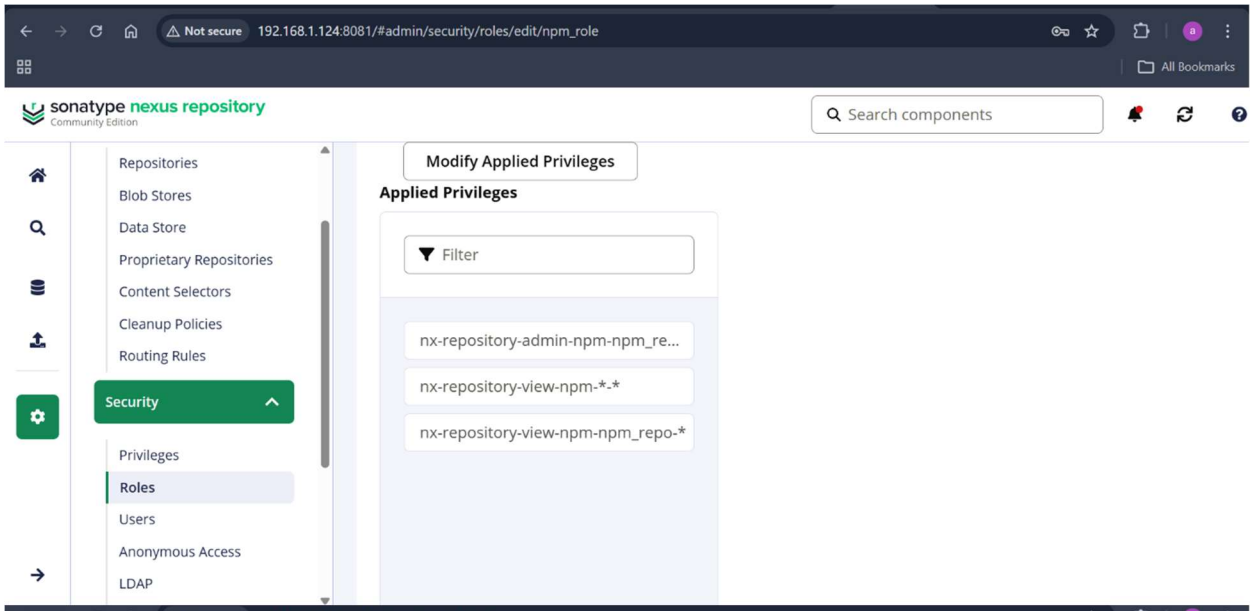


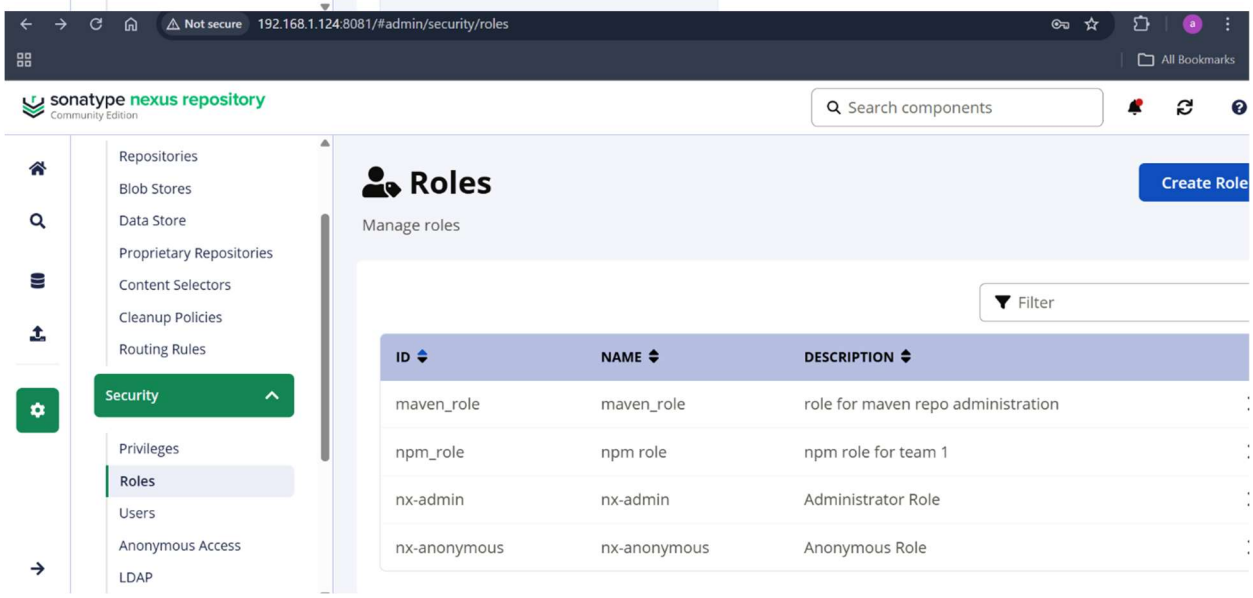
5. Team 1 User Configuration

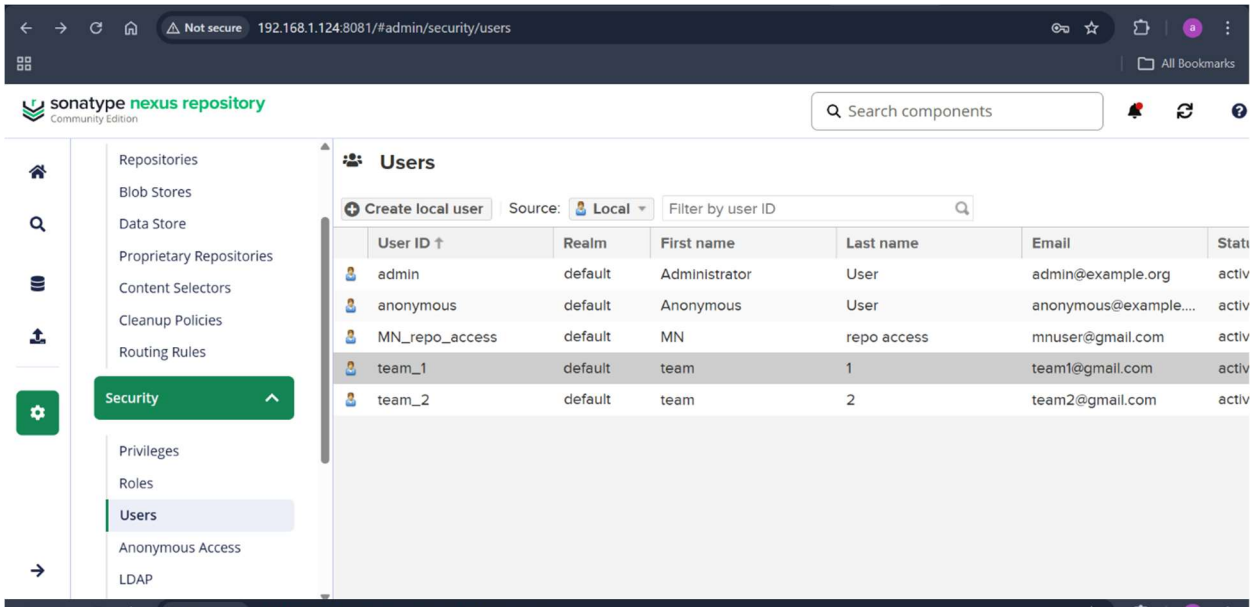
A Nexus user account was created for the **Project 1 development team**.

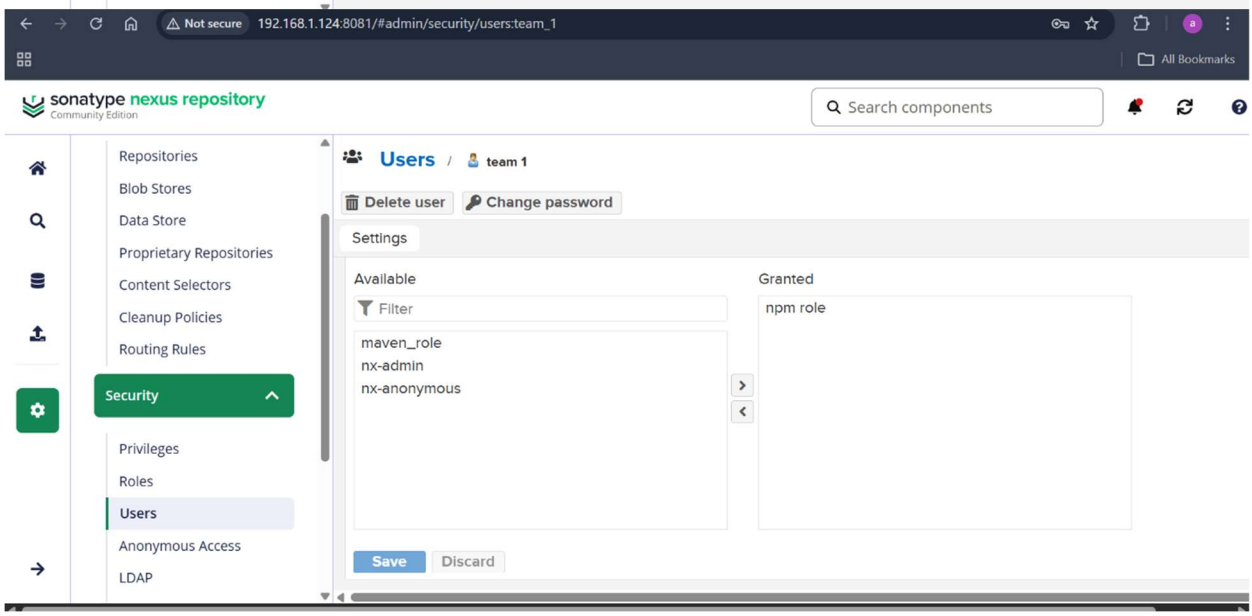
This user was granted permissions to:

- Publish NodeJS packages
- Read packages from the npm repository

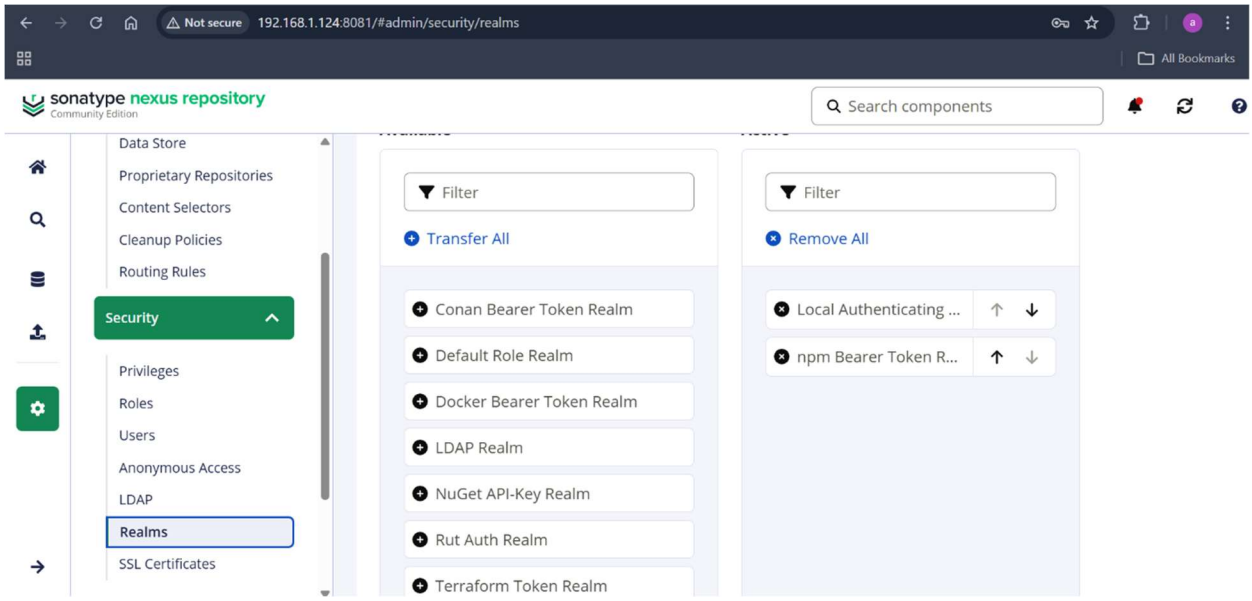






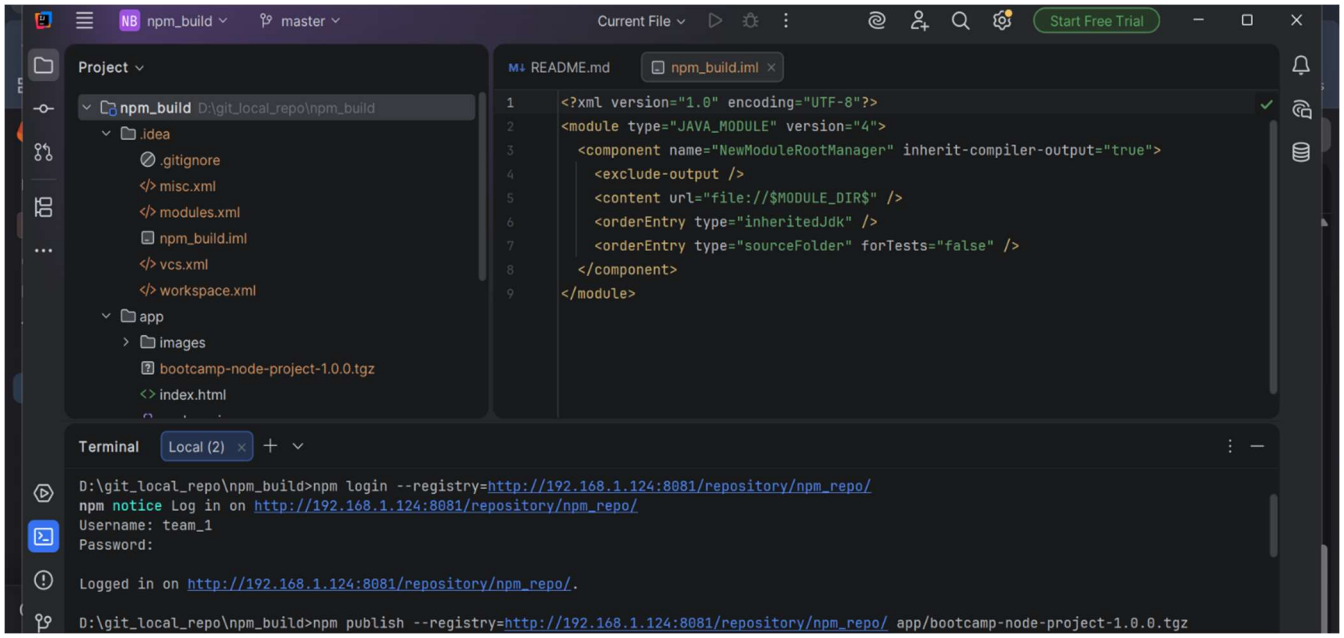


To login or publish npm artifacts, I had to also navigate to **settings – security – realms** and enable 'npm Bearer Token Realm'.



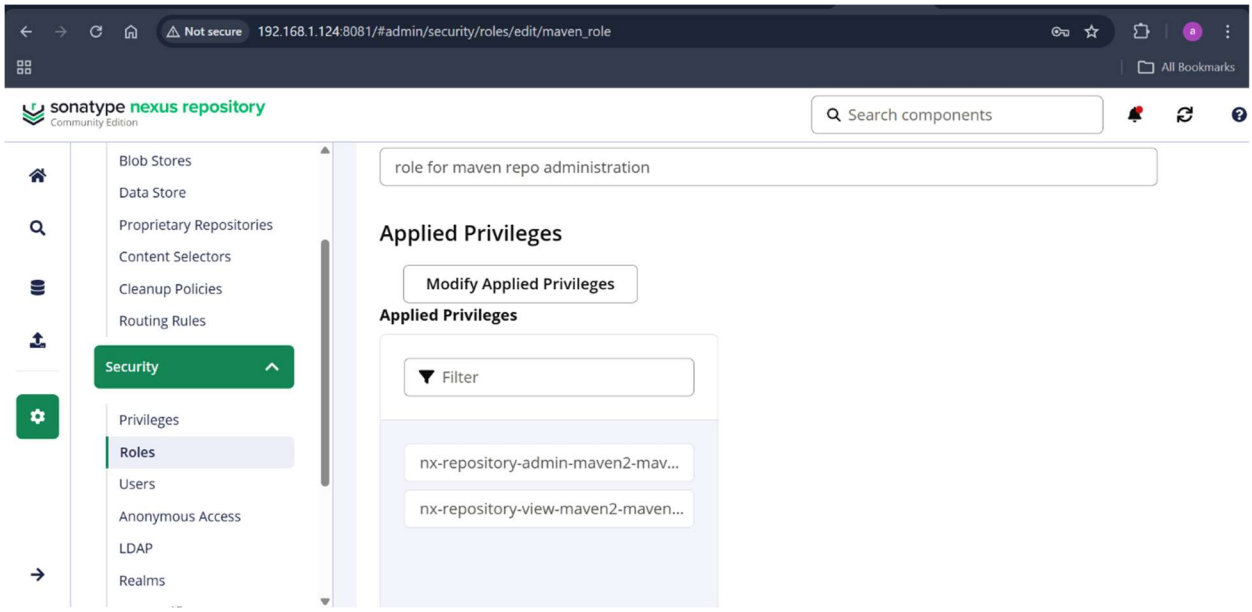
6. Publishing NodeJS Artifacts

The NodeJS application developed was packaged and published to the npm repository to verify correct repository permissions and functionality.



7. Maven Repository Setup

A **Maven hosted repository** was created to store Java artifacts produced by the Gradle build system.



8. Team 2 User Configuration

A separate user account was created for the **Project 2 development team**.

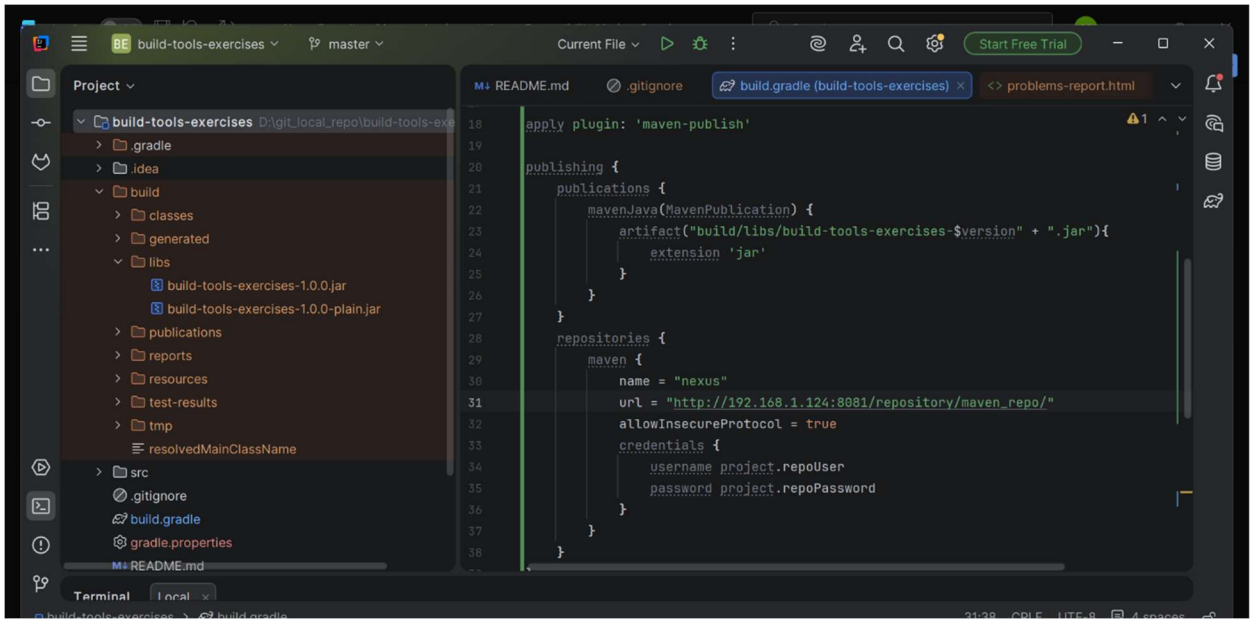
This user was granted permissions to:

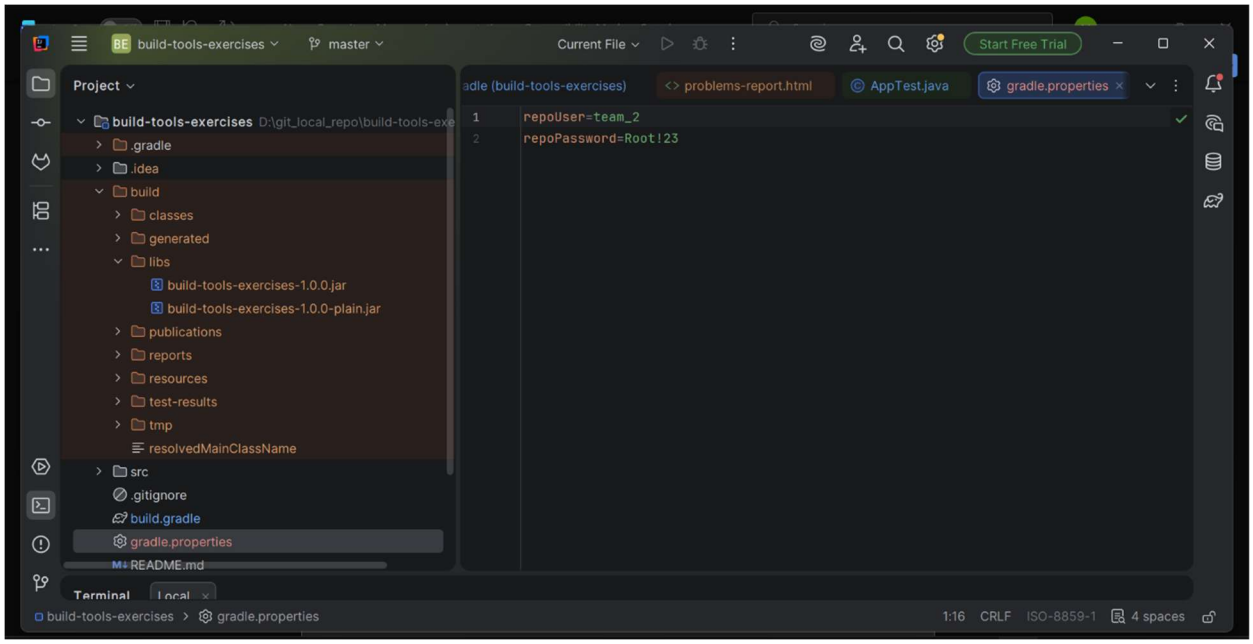
- Upload artifacts to the Maven repository
- Download artifacts from the repository

9. Publishing Java Artifacts

The Java application from the Build Tools module was compiled using **Gradle**.

The build process generated a **JAR artifact**, which was then published to the Nexus Maven repository.





10. Retrieving Artifacts via Nexus REST API

The Nexus REST API was used to query the repository and retrieve metadata about the latest NodeJS artifact.

The API response returns JSON data containing:

- artifact name
- version
- repository
- download URL

```
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020: /opt
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020:/opt$ curl -u team_1:'Root!23' -X GET 'h
ttp://192.168.1.124:8081/service/rest/v1/components?repository=npm_repo&sort=version'
{
  "items" : [ {
    "id" : "bnBtX3JlcG86NGYxYmJjZGQ",
    "repository" : "npm_repo",
    "format" : "npm",
    "group" : "",
    "name" : "bootcamp-node-project",
    "version" : "1.0.0",
    "assets" : [ {
      "downloadUrl" : "http://192.168.1.124:8081/repository/npm_repo/bootcamp-node-pro
ject/-/bootcamp-node-project-1.0.0.tgz",
      "path" : "/bootcamp-node-project/-/bootcamp-node-project-1.0.0.tgz",
      "id" : "bnBtX3JlcG86NGYxYmJjZGQ",
      "repository" : "npm_repo",
      "format" : "npm",
      "checksum" : {
        "sha1" : "1faf0cafea16476e7d5d20e0398b56d1dc0f18ba",
        "md5" : "ebb15462879ab4fecf41110f63806dfc"
      },
      "contentType" : "application/octet-stream",
      "lastModified" : "2026-03-05T01:07:13.839+00:00",
      "lastDownloaded" : "2026-03-06T00:09:59.416+00:00",
      "uploader" : "team_1",
```

```
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020: ~
ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020:~$ wget --user='team_1' --password='Root!23' "http://192.168.1.12
4:8081/repository/npm_repo/bootcamp-node-project/-/bootcamp-node-project-1.0.0.tgz"
--2026-03-09 19:11:33--  http://192.168.1.124:8081/repository/npm_repo/bootcamp-node-project/-/bootcamp-node-pr
oject-1.0.0.tgz
Connecting to 192.168.1.124:8081... connected.
HTTP request sent, awaiting response... 401 Unauthorized
Authentication selected: BASIC realm="Sonatype Nexus Repository Manager"
Reusing existing connection to 192.168.1.124:8081.
HTTP request sent, awaiting response... 200 OK
Length: 79328 (77K) [application/octet-stream]
Saving to: ‘bootcamp-node-project-1.0.0.tgz.1’

bootcamp-node-project-1.0.0 100%[===========================================>]  77.47K  --.-KB/s    in 0s

2026-03-09 19:11:33 (541 MB/s) - ‘bootcamp-node-project-1.0.0.tgz.1’ saved [79328/79328]

ali-abdul-aleem@ali-abdul-aleem-OptiPlex-3020:~$
```

11. Deployment Automation

A **Bash script** was developed to automate the process of retrieving the latest artifact from Nexus and running the application.

The script performs the following tasks:

1. Fetch artifact metadata using the Nexus API
2. Extract the download URL
3. Download the artifact
4. Extract the package
5. Start the NodeJS application

(Automation scripts available in folder “scripts”)